Chapter 5
Transport layer

ONE LOVE. ONE FUTURE.

1

1

Contents

- Principles of transport layer
- UDP protocol
- Reliable data transfer
- TCP protocol

2

Basic concepts

ONE LOVE. ONE FUTURE.

3

3

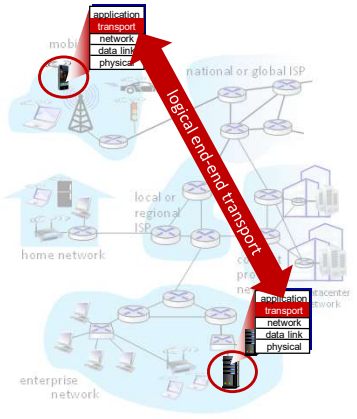
Network layer in the TCP/IP architecture

Application (HTTP, Mail, …)	Support network applications
Transport (UDP, TCP …)	Control data transmission between processes at the application layer
Network (IP, ICMP…)	Controlling data transmission to another network
Datalink (Ethernet, ADSL…)	Facilitating communication for subsequent components on the same network
Physical (bits…)	Transmitting and receiving bit streams on the physical transmission line

4

Transport services and protocols

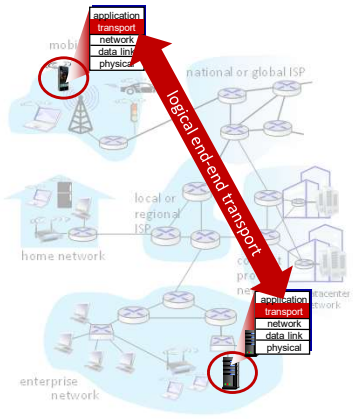
- Provide *logical communication* between application processes running on different hosts
- Sender:
  - Receives data from application player
  - Breaks application messages into segments, passes to network layer
  - If data is too large, splits to smaller parts and puts to different datagrams
- Receiver:
  - Receive data from network layer
  - Reassembles segments into messages, passes to application layer



5

Transport services and protocols

- Install on end-to-end hosts
  - No installation on routers or switches
- Two transport protocols available to Internet applications
  - Reliable and connection oriented. For example: TCP
  - Unreliable and connectionless. For example: UDP



6

Transport vs. network layer services and protocols

household analogy:

12 kids in Ann's house sending letters to 12 kids in Bill's house:

- hosts = houses
- processes = kids
- app messages = letters in envelopes

7

Transport vs. network layer services and protocols

- network layer: logical communication between *hosts*
- transport layer: logical communication between *processes*
  - relies on, enhances, network layer services

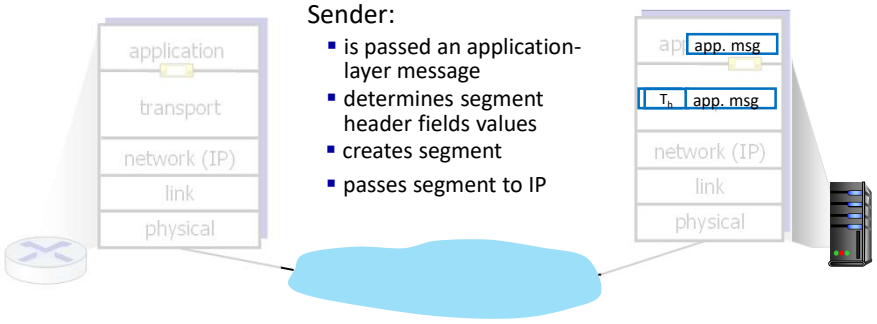
household analogy:

12 kids in Ann's house sending letters to 12 kids in Bill's house:

- hosts = houses
- processes = kids
- app messages = letters in envelopes

8

Transport Layer Actions

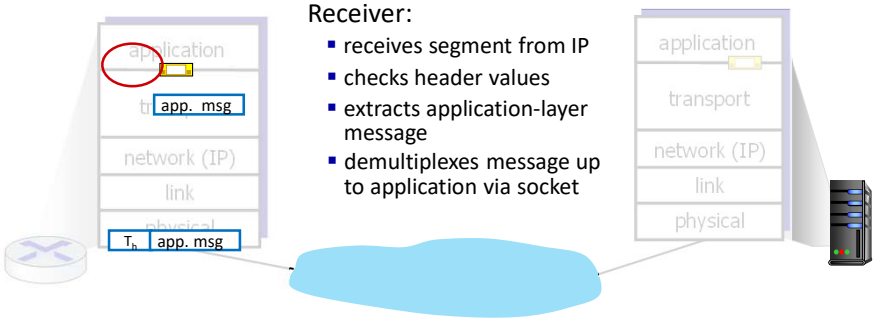


Sender:

- is passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP

9

Transport Layer Actions



Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket

10

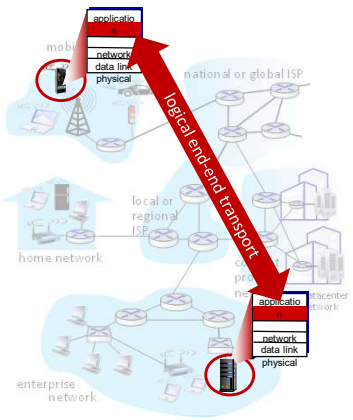
Why there are two kind of services?

- Various requirements about services from applications
- Applications that need 100% reliable data transfer, e.g. FTP, Mail…
  - Uses TCP (reliable) as transport services
- Application that need fast data transfer but can tolerate with packet lost, e.g. VoIP, Video Streaming
  - Uses UDP (best-effort) as transport services

11

Two principal Internet transport protocols

- TCP: Transmission Control Protocol
  - reliable, in-order delivery
  - congestion control
  - flow control
  - connection setup
- UDP: User Datagram Protocol
  - unreliable, unordered delivery
  - no-frills extension of “best-effort” IP
- services not available:
  - delay guarantees
  - bandwidth guarantees



12

Applications and transport services

Application	Application protocols	Transport protocols
e-mail	SMTP	TCP
remote terminal access	Telnet	TCP
Web	HTTP	TCP
file transfer	FTP	TCP
streaming multimedia	Specific protocols (e.g. RealNetworks)	TCP or UDP
Internet telephony	Specific protocols (e.g., Vonage,Dialpad)	Usually UDP

13

Functionalities

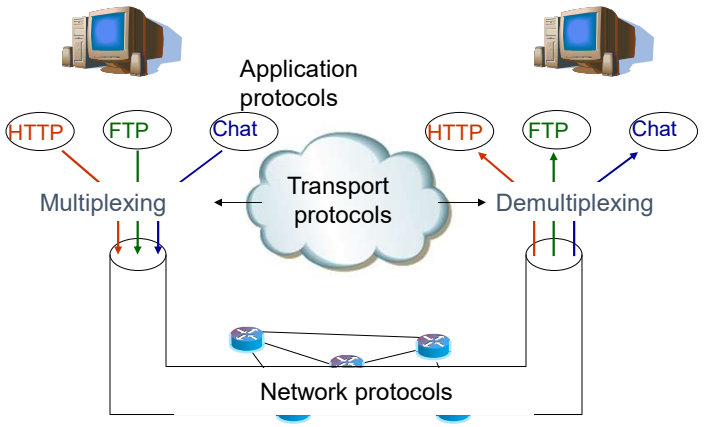
MUX/DEMUX

Error control

ONE LOVE. ONE FUTURE.

14

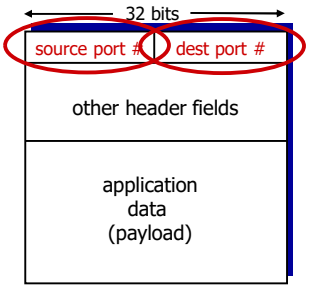
Mux/Demux



15

How demultiplexing works

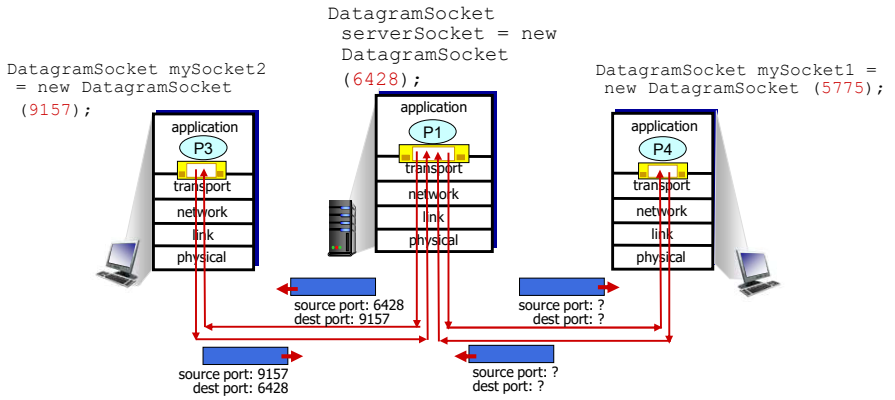
- At network layer, IP packet was identified by IP address
  - To distinguish the hosts
- How to distinguish different networking applications in the same host
  - Use port number (16-bit)
  - Each process is assigned a unique port number
- Socket: a couple of *IP addresses & port numbers*
- Host uses IP addresses & port numbers to direct segment to appropriate socket



TCP/UDP segment format

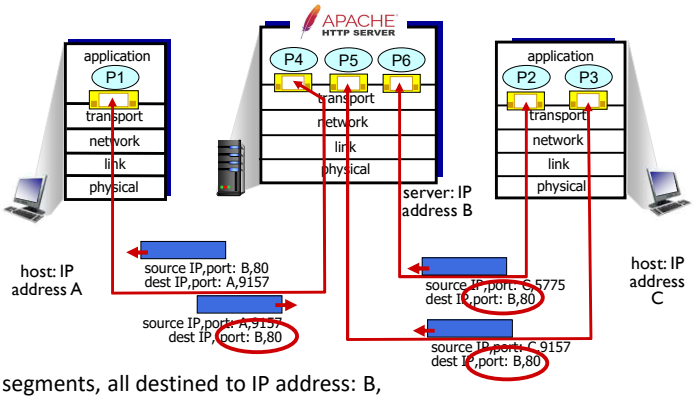
16

Connectionless demultiplexing: an example



18

Connection-oriented demultiplexing: example



Three segments, all destined to IP address: B, dest port: 80 are demultiplexed to *different* sockets

20

Error Control

- Use CRC or Checksum
- Checksum
  - Similar as checksum (16 bits) of IP
- Mechanism
  - Split data to 16-bit chunks
  - These chunks are then added, any generated carry is added back to the sum
  - Then, the 1's complement of the sum is performed and put in the checksum field

Example of checksum

	+ 1
	01110111
Frame 3:	+ 11110000
Partial Sum:	1 01100111
	+ 1
	01101000
Frame 4:	+ 11000011
Partial Sum:	1 00101011
	+ 1
Sum:	00101100
Checksum:	11010011

	+ 1
	01110111
Frame 3:	+ 11110000
Partial Sum:	1 01100111
	+ 1
	01101000
Frame 4:	+ 11000011
Partial Sum:	1 00101011
	+ 1
Sum:	00101100
Checksum:	11010011

Reliable data transfer

ONE LOVE. ONE FUTURE.

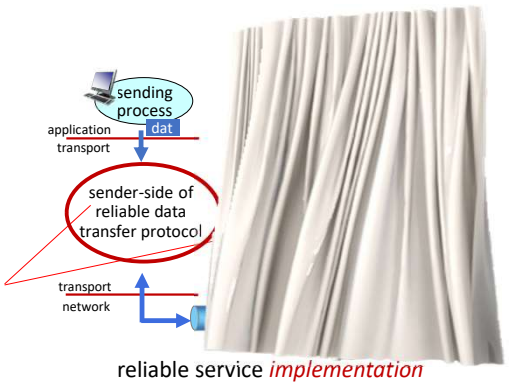
24

24

Principles of reliable data transfer

Sender, receiver do *not* know the "state" of each other, e.g., was a message received?

- unless communicated via a message



reliable service *implementation*

SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

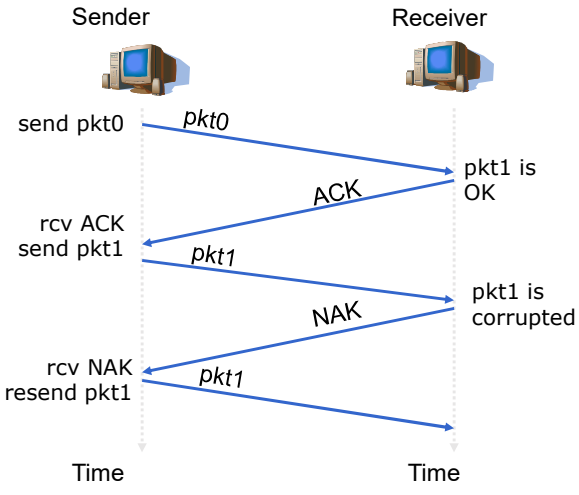
25

25

Reliable data transfer

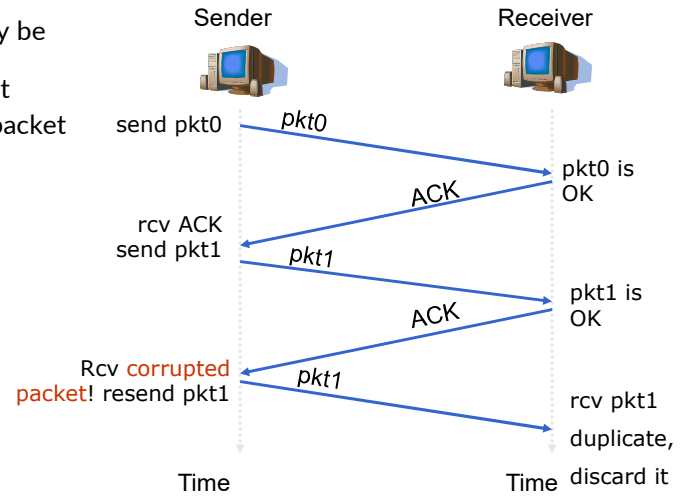
- How to detect error?
  - Checksum
- How to inform sender?
  - ACK (*acknowledgements*):
  - NAK (*negative acknowledgements*): tell sender that pkt has error
- Reaction of sender?
  - Retransmit the error packet once received NAK

Error control



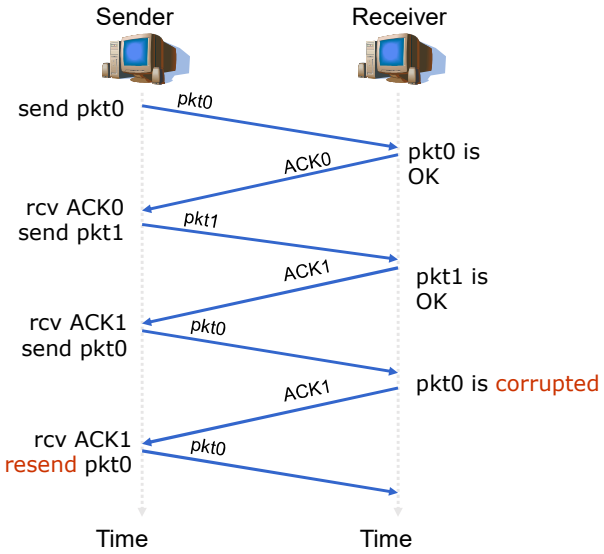
Error in ACK/NAK

- ACK/ NAK may be corrupted
- Packet is resent
- How to solve packet repetition?
- Use Seq.#



28

Error control without NAK



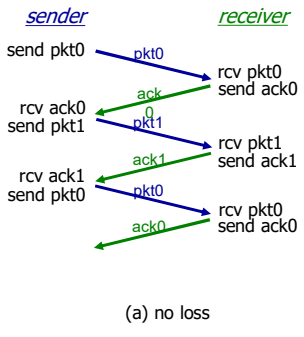
29

Chanel with error and packet lost

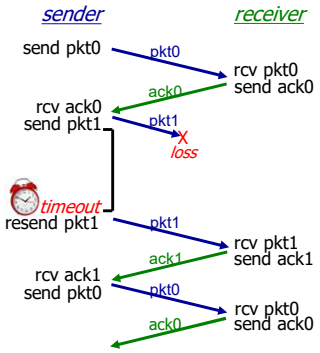
- Data and ACK can be lost
  - If no ACK is received? How sender knows and decides to resend data?
  - Sender should wait for ACK for a certain time. Timeout!
- How long should be timeout?
  - At least 1 RTT (Round Trip Time)
  - Need to start a timer each time sending a packet
- What if packet arrives and ACK is lost?
  - Packet should be numbered.

30

Illustration



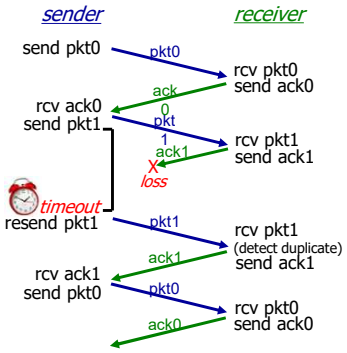
(a) no loss



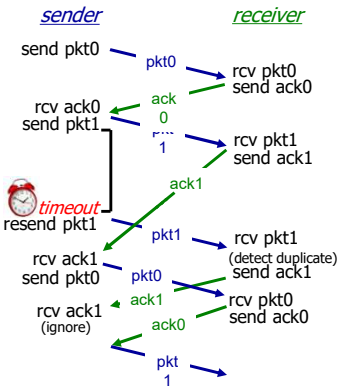
(b) packet loss

31

Illustration (2)



(c) ACK loss



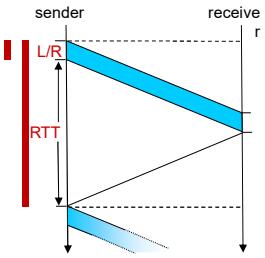
(d) premature timeout/ delayed ACK

Performance of reliable data transfer (stop-and-wait)

- U_{sender}: *utilization* – fraction of time sender busy sending
- example: 1 Gbps link, 15 ms prop. delay, 8000 bit packet
  - time to transmit packet into channel:

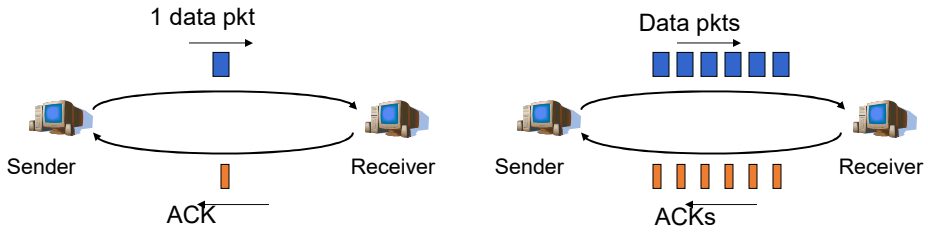
$$D_{trans} = \frac{L}{R} = \frac{8000\ bits}{10^9\ bits/sec} = 8\ microsecs$$

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$



- The performance stinks!
- Protocol limits performance of underlying infrastructure (channel)

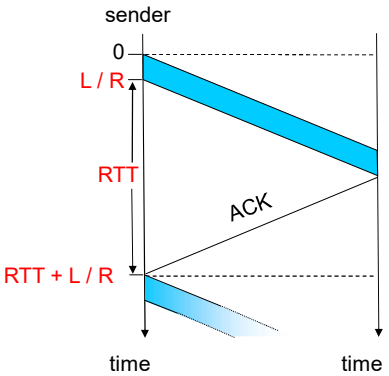
Transmission in pipeline



34

Comparison of efficiency

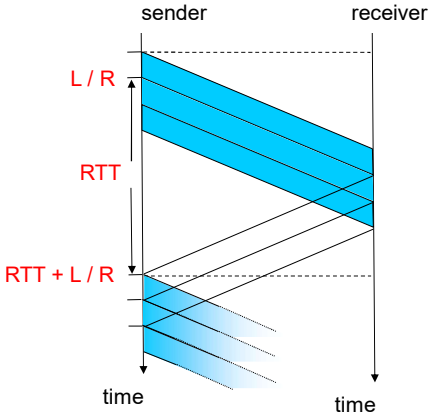
stop-and-wait



L: Size of data pkt
R: Link bandwidth
RTT: Round trip time

$$\text{Performance} = \frac{L / R}{RTT + L / R}$$

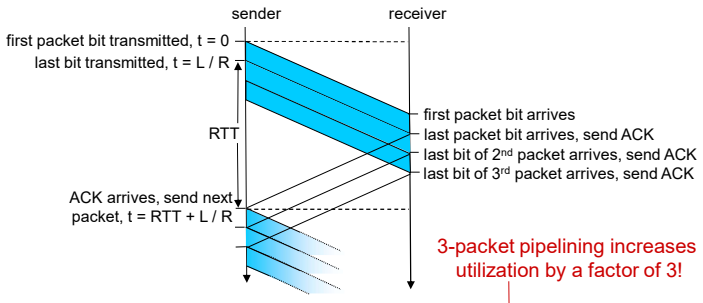
Pipeline



$$\text{Performance} = \frac{3 * L / R}{RTT + L / R}$$

35

Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Sliding windows: mechanism

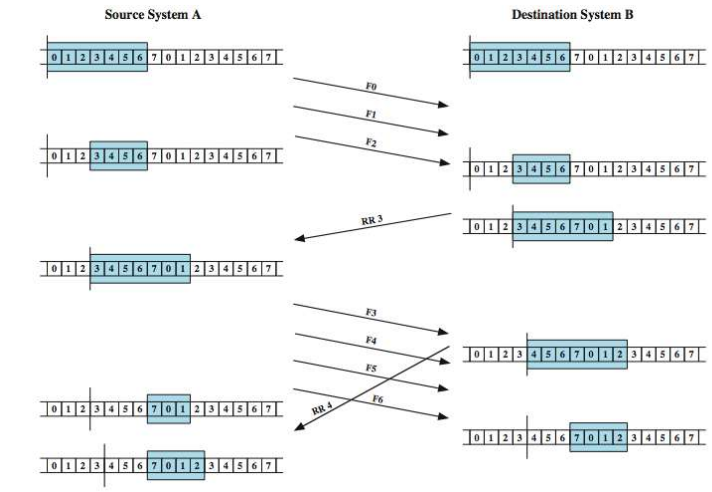
- Send multiple segments/frames simultaneously to reduce the waiting time
- Store transmitted frames while waiting for ACKs
- The number of transmitted frames is dependent on buffer
- After receiving ACK
  - Release the acknowledged (ACK) frame from the buffer
  - Send the next frame

Sliding windows: mechanism

- Consider two stations A and B connected by a simplex transmission line.
  - B has a buffer for n data frames.
  - Thus, B can receive n data frames simultaneously without the need for acknowledgment.
- Acknowledgment:
  - To 'remember' the acknowledged frames, it is necessary to number the frames.
  - B acknowledges a frame by indicating the frame number B is waiting to receive, implicitly acknowledging all frames received before.
  - One acknowledgment can be used for multiple data frames.

38

Sliding window mechanism

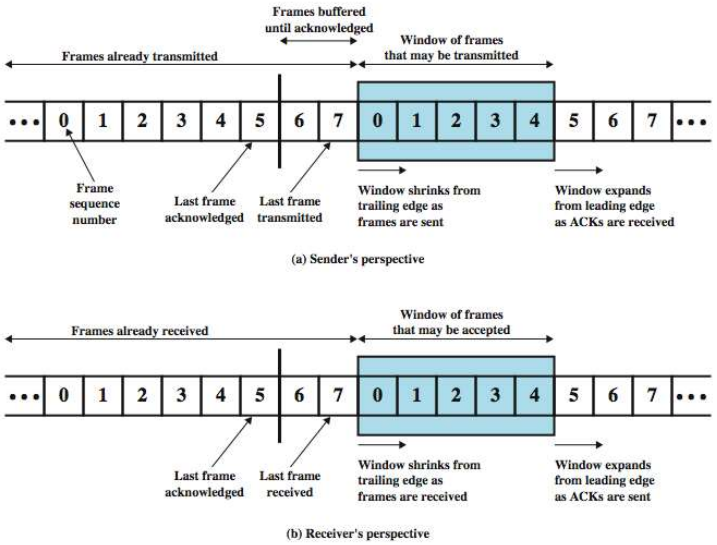


In the window are frames waiting to be sent

In the window are frames waiting to be received

39

Sliding window mechanism



40

Sliding window mechanism

- The frames being sent are numbered.
  - The sequence number must be greater than or equal to the window size.
- Acknowledgments for received frames are sent with numbered acknowledgments.
- Acknowledgments are cumulative.
  - If frames 1, 2, 3, 4 are successfully received, only acknowledgment for frame 4 is sent.
- Upon receiving an acknowledgment for frame k, it implies that all frames k-1, k-2, and so on, have been successfully received.

41

Sliding window mechanism

- Source managment
  - Frames sent successfully.
  - Frames sent but not acknowledged.
  - Frames that can be sent immediately.
  - Frames that cannot be sent immediately.
- Destination management
  - Frames received.
  - Frames waiting to be received.

42

Acknowledgment and Retransmission Techniques

- **ARQ automatic repeat request**
- There are 3 standardized versions
  - Stop-and-Wait ARQ
    - Previously presented
  - Go-Back-N ARQ
  - Selective Reject/ Selective Repeat ARQ

43

Go-Back-N: sender

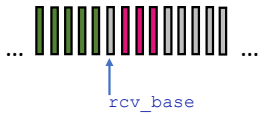
- sender: "window" of up to N, consecutive transmitted but unACKed pkts
  - k-bit seq # in pkt header

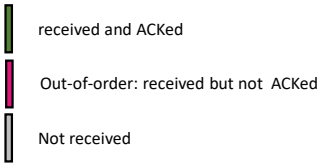
- *cumulative ACK:* ACK(*n*): ACKs all packets up to, including seq # *n*
  - on receiving ACK(*n*): move window forward to begin at *n+1*
- timer for oldest in-flight packet
- *timeout(n):* retransmit packet n and all higher seq # packets in window

Go-Back-N: receiver

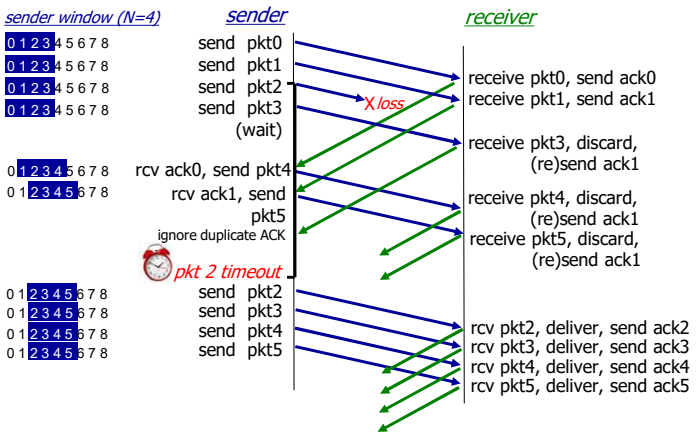
- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
  - may generate duplicate ACKs
  - need only remember `rcv_base`
- on receipt of out-of-order packet:
  - can discard (don't buffer) or buffer: an implementation decision
  - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:





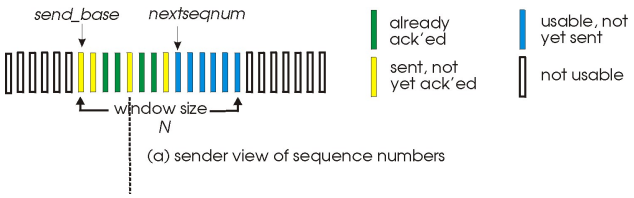
Go-Back-N in action



Selective repeat

- receiver *individually* acknowledges all correctly received packets
  - buffers packets, as needed, for eventual in-order delivery to upper layer
- sender times-out/retransmits individually for unACKed packets
  - sender maintains timer for each unACKed pkt
- sender window
  - *N* consecutive seq #s
  - limits seq #s of sent, unACKed packets

Selective repeat: sender, receiver windows



48

Selective repeat: sender and receiver

sender

data from above:

- if next available seq # in window, send packet

timeout(*n*):

- resend packet *n*, restart timer

ACK(*n*) in [sendbase,sendbase+N]:

- mark packet *n* as received
- if n smallest unACKed packet, advance window base to next unACKed seq #

receiver

packet *n* in [rcvbase, rcvbase+N-1]

- send ACK(*n*)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

packet *n* in [rcvbase-N,rcvbase-1]

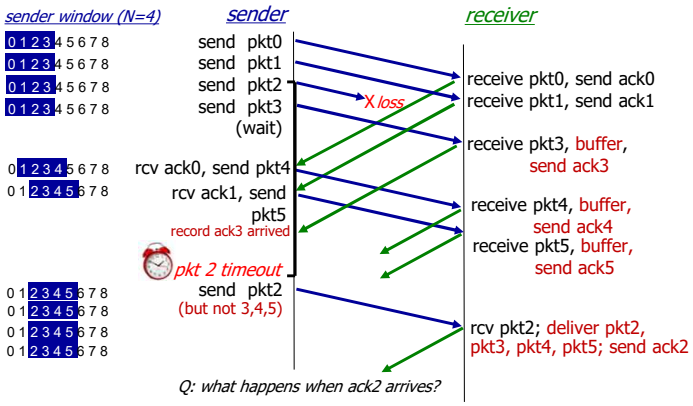
- ACK(*n*)

otherwise:

- ignore

49

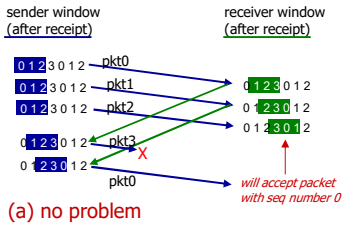
Selective Repeat in action



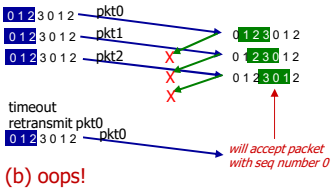
Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3



(a) no problem



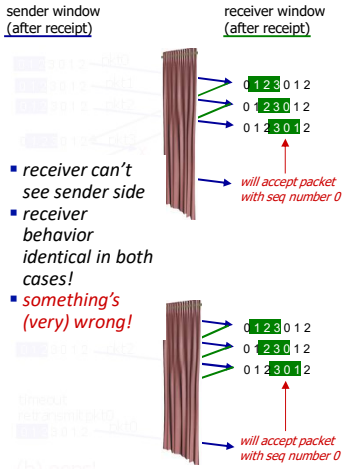
(b) oops!

Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

Q: what relationship is needed between sequence # size and window size to avoid problem in scenario (b)?



52

UDP
User Datagram Protocol

ONE LOVE. ONE FUTURE.

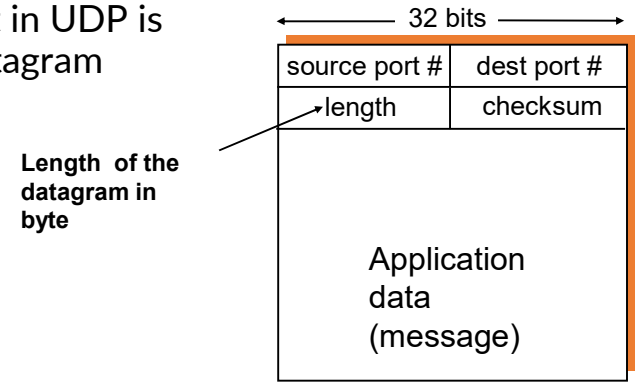
53

"Best effort" protocols

- Why UDP?
  - No need to establish connection (cause delay)
  - Simple
  - Small header
  - No congestion control → send data as fast as possible
- Main functionality of UDP?
  - MUX/DEMUX
  - Detect error by checksum

Datagram format

- Data unit in UDP is called datagram



Issues of UDP

- No congestion control
  - Cause overload of the Internet
- No reliability
  - Applications have to implement themselves mechanisms to control errors

58

Summary: UDP

- "no frills" protocol:
  - segments may be lost, delivered out of order
  - best effort service: "send and hope for the best"
- UDP has its plusses:
  - no setup/handshaking needed (no RTT incurred)
  - can function when network service is compromised
  - helps with reliability (checksum)
- build additional functionality on top of UDP in application layer (e.g., HTTP/3)

59

TCP
Transmission Control Protocol

TCP segment structure
Connection management
Flow control
Congestion control

ONE LOVE. ONE FUTURE.

60

60

Overview of TCP

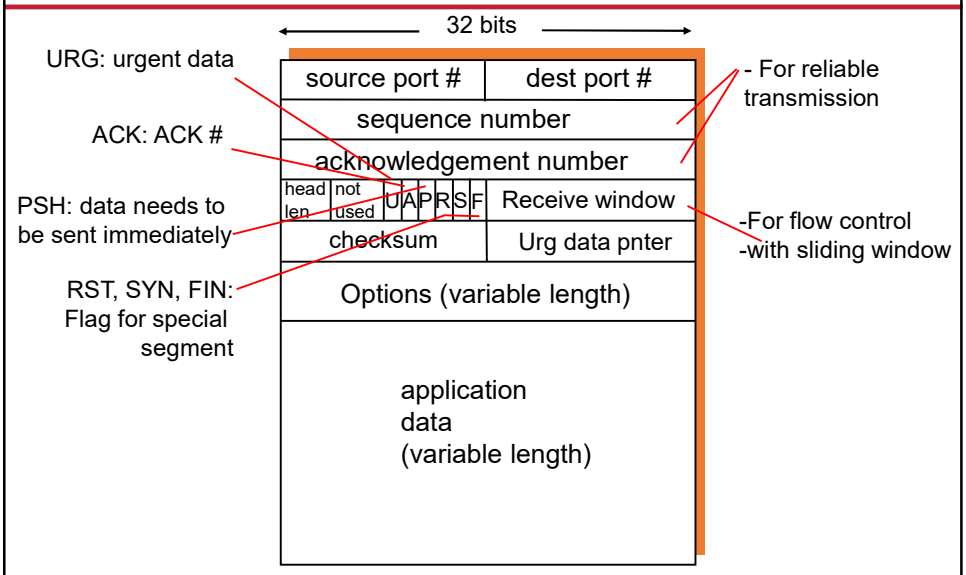
- Connection oriented
  - 3 steps hand-shake
- Data transmission in stream of byte, reliable
  - Use buffer
- Transmit data in pipeline
  - Increase the performance
- Flow control
  - Sliding windows
- Congestion control
  - Detect congestion and solve

SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

61

61

TCP segment



62

How TCP provide reliable service?

- In order to assure if data arrives to destination:
  - Seq. #
  - Ack
- TCP cycle life:
  - Connection establishing
    - 3 steps
  - Data transmission
  - Close connection

63

Acknowledgment Mechanism in TCP

Sequence numbers:

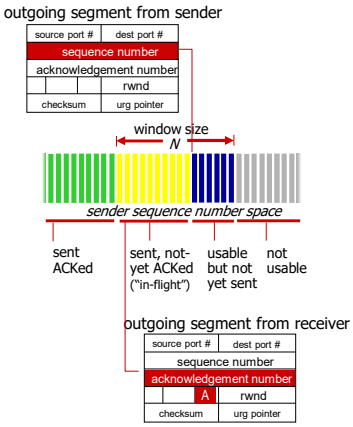
- byte stream "number" of first byte in segment's data

Acknowledgements:

- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

- A: TCP spec doesn't say, - up to implementor



64

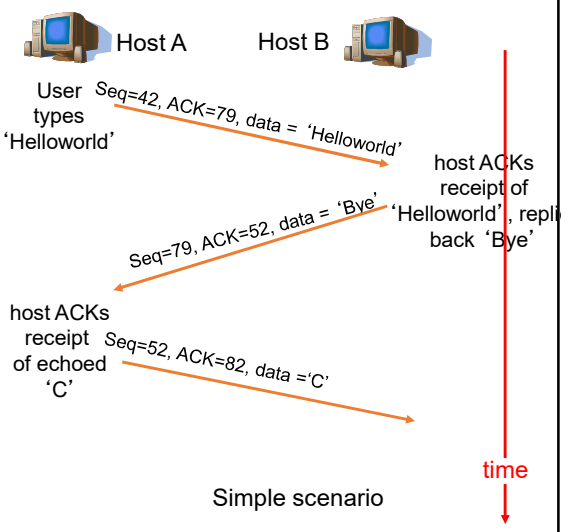
Acknowledgment Mechanism in TCP

Seq. #:

- Sequence number of the first byte in the message stream

ACK:

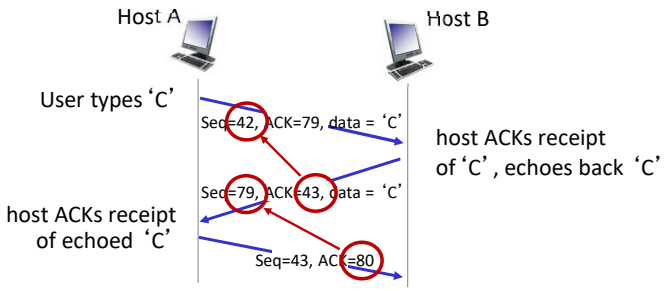
- Desired sequence number of the byte to be received from the peer.
- Implicitly acknowledges successful receipt of previous bytes



Simple scenario

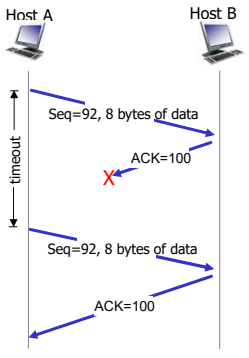
65

Acknowledgment Mechanism in TCP

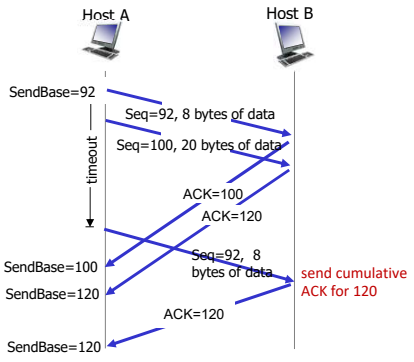


simple telnet scenario

TCP: retransmission scenarios

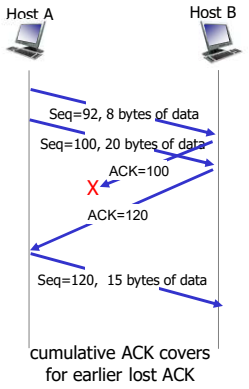


lost ACK scenario



premature timeout

TCP: retransmission scenarios



68

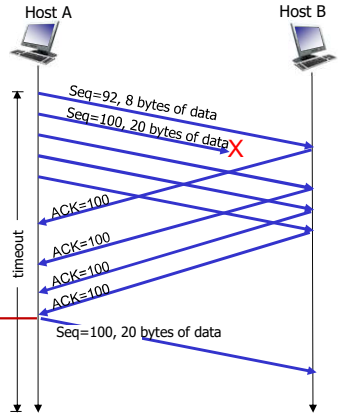
TCP fast retransmit

TCP fast retransmit

if sender receives 3 additional ACKs for same data ("triple duplicate ACKs"), resend unACKed segment with smallest seq #

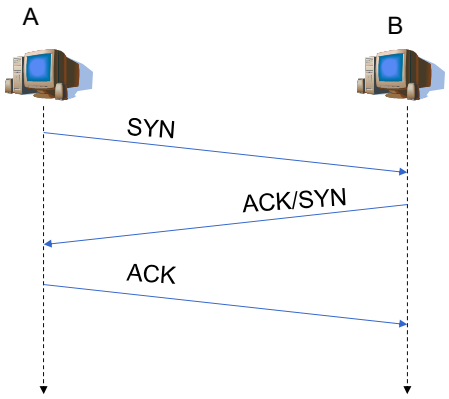
- likely that unACKed segment lost, so don't wait for timeout

Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!



69

Connection establishing in TCP: 3 steps (3-way handshake)

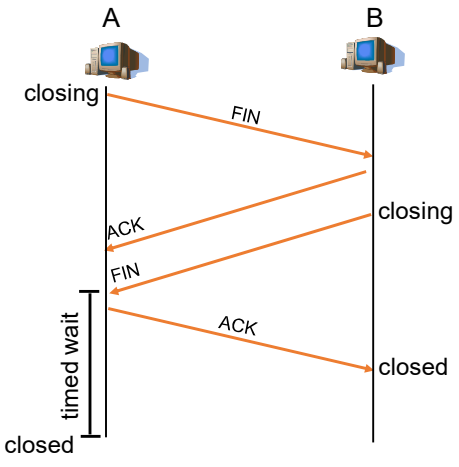


- Step 1: A sends SYN to B
  - Indicate initial value of seq # of A
  - No data
- Step 2: B receives SYN, replies by SYNACK
  - B initiates the buffer on its side
  - Indicate initial value of seq. # of B
- Step 3: A receives SYNACK, replies ACK, maybe with data.

70

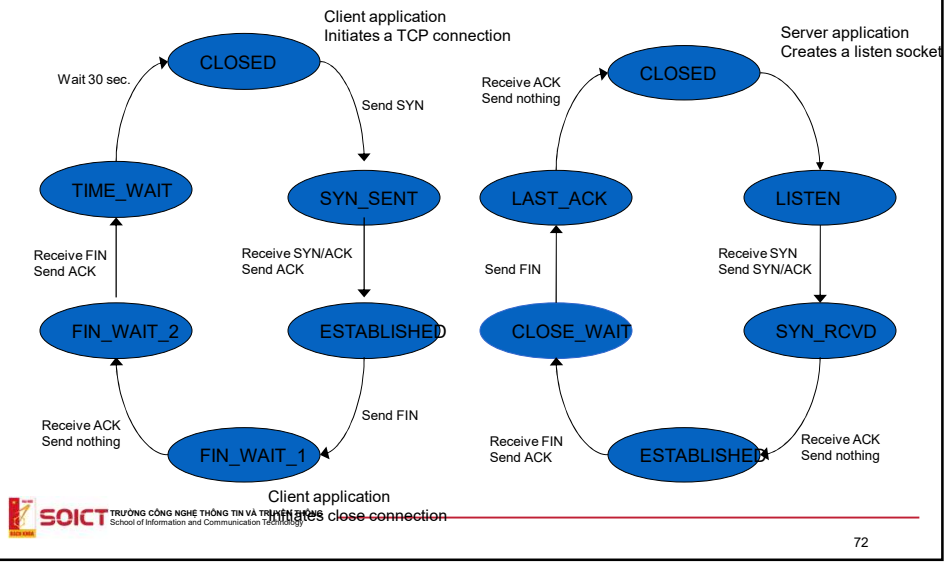
Close connection

- Step 1: Send FIN to B
- Step 2: B receives FIN, replies ACK, closes the connection and sends FIN.
- Step 3: A receives FIN, replies ACK, go to "waiting".
- Bước 4: B receives ACK. close connection



71

Simplified life cycle of TCP



72

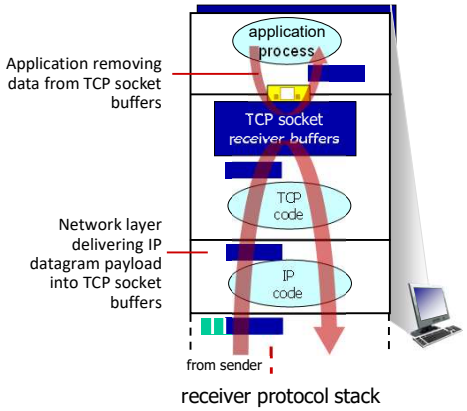
Flow control in TCP

ONE LOVE. ONE FUTURE.

73

TCP flow control

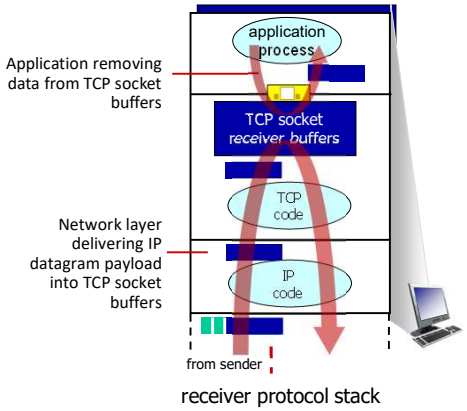
Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



74

TCP flow control

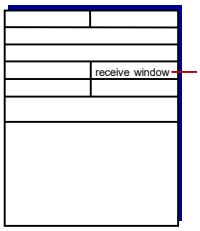
Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



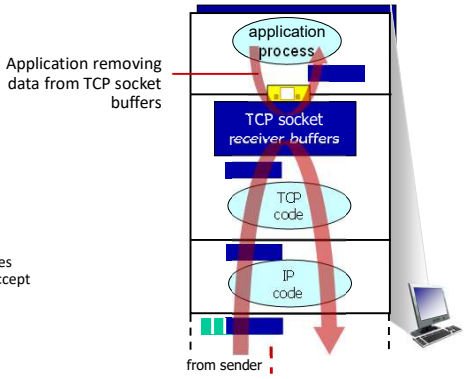
75

TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?



flow control: # bytes receiver willing to accept



receiver protocol stack

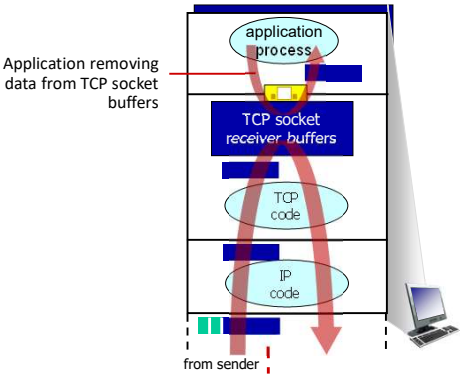
76

TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

flow control

receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast



receiver protocol stack

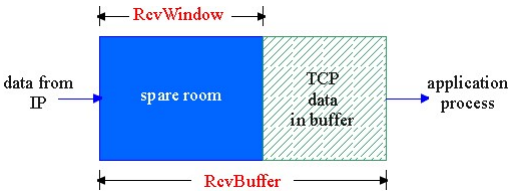
77

TCP flow control

- Control the amount of data sent.
  - Ensure efficiency is optimal.
  - Avoid overloading parties.
- Parties will have control windows.
  - Rwnd: Receive window.
  - CWnd: Congestion window.
- The amount of data sent must be less than min(Rwnd, CWnd).

78

TCP flow control



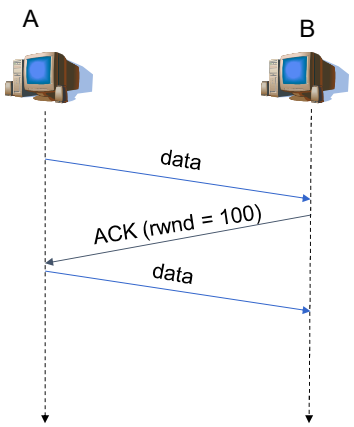
- Size of free buffer

= Rwnd

= **RcvBuffer-[LastByteRcvd - LastByteRead]**

79

Information exchanged on Rwnd



- Receiver inform regularly to senders the value of `Rwnd` in acknowledgment segments

80

Congestion control in TCP

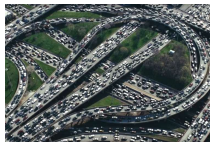
ONE LOVE. ONE FUTURE.

81

Principles of congestion control

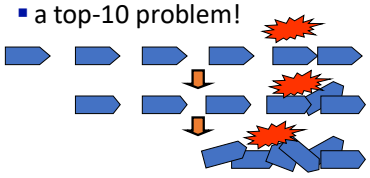
Congestion:

- informally: "too many sources sending too much data too fast for *network* to handle"
- manifestations:
  - long delays (queueing in router buffers)
  - packet loss (buffer overflow at routers)
  - Network condition becomes worse



congestion control: too many senders, sending too fast

- different from flow control!
- a top-10 problem!



Congestion occur

flow control: one sender too fast for one receiver

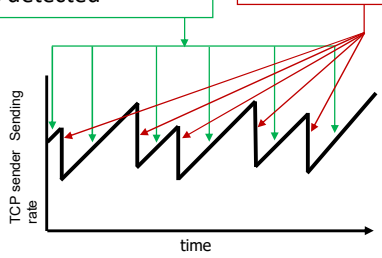
82

TCP congestion control: AIMD

- *approach:* senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase
increase sending rate by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease
cut sending rate in half at each loss event



AIMD sawtooth behavior: *probing* for bandwidth

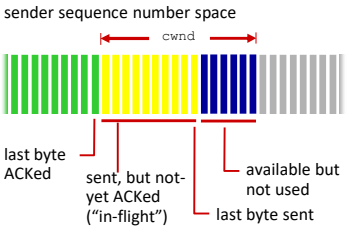
83

TCP AIMD: more

Multiplicative decrease detail: sending rate is
- Cut in half on loss detected by triple duplicate ACK (TCP Reno)
- Cut to 1 MSS (maximum segment size) when loss detected by timeout (TCP Tahoe)

Why AIMD?
- AIMD – a distributed, asynchronous algorithm – has been shown to:
  - optimize congested flow rates network wide!
  - have desirable stability properties

TCP congestion control: details



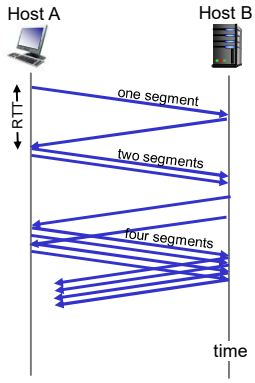
TCP sending behavior:
- *roughly:* send `cwnd` bytes, wait RTT for ACKS, then send more bytes

$$\text{TCP rate} \approx \frac{\texttt{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

- TCP sender limits transmission: `LastByteSent- LastByteAcked` ≤ `cwnd`
- `cwnd` is dynamically adjusted in response to observed network congestion (implementing TCP congestion control)

TCP slow start

- when connection begins, increase rate exponentially until first loss event:
  - initially `cwnd` = 1 MSS
  - double `cwnd` every RTT
  - done by incrementing `cwnd` for every ACK received
- *summary:* initial rate is slow, but ramps up exponentially fast



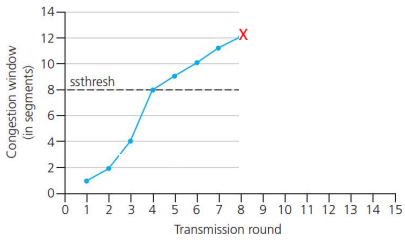
TCP: from slow start to congestion avoidance

Q: when should the exponential increase switch to linear?

A: when `cwnd` gets to 1/2 of its value before timeout.

Implementation:

- variable `ssthresh`
- on loss event, `ssthresh` is set to 1/2 of `cwnd` just before loss event

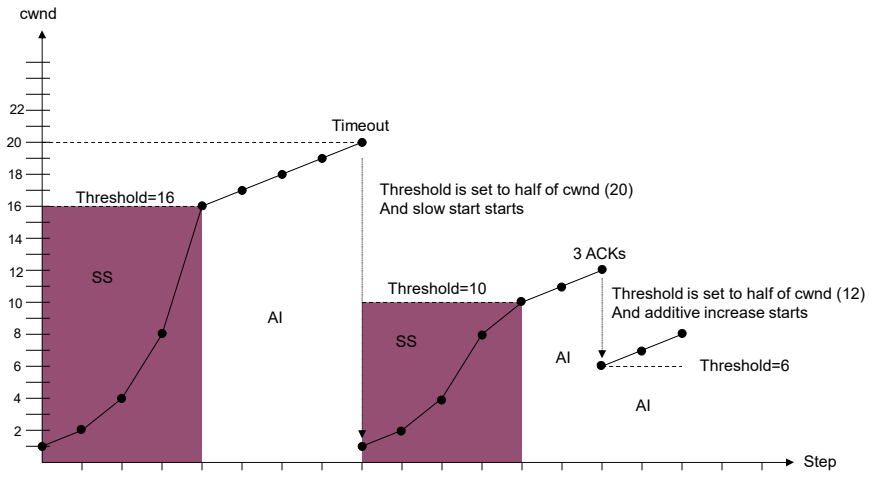


Congestion control

- Timeout on the sender side:
  - TCP sets the threshold (ssthresh) to half of the current value of cwnd.
  - TCP sets cwnd to 1 MSS.
  - TCP transitions to slow start.
- If receiving 3 identical ACKs:
  - TCP sets the threshold to half of the current value of cwnd.
  - TCP transitions to congestion avoidance state.

88

Congestion control – illustration



89

Exercise

- Assume that we need transmit 1 file
  - File size *O* =100KB over TCP connection
  - S is the size of each TCP segment, *S* = 536 byte
  - *RTT = 3 ms*.
- Assume that the congestion window size of TCP is fixed with value W.
  - What is the transmission time if W=1 (stop-and-wait)?
  - What is the transmission time if W=7?
  - What is the minimum transmission time?

  The transmission speed is
  - R = 20 Mbit/s; and R = 100 Mbits/s.

Solution (cont.)

- T transmit (W packet) = W * S/R
- Transmit without waiting:
- => (W-1)*S/R >= RTT
- => W >= RTT*R/S +1
- Time to transmit all data L = L/R + RTT
- R=100 Mbps
  - W>= 100ms * 100 Mbps/ (536*8) + 1